



Object Oriented Programming with Java

CHAP 1: JAVA PROGRAMMING BASICS

Fundamentals, Programming Basics & Controls Structures: By Larisse Mahoro



1.1. Java Language Fundamentals



Overview

- Java is a platform as well as a language
- The Java language was developed by James Gosling at Sun Microsystems (which is now a subsidiary of Oracle Corporation) and first released in 1995
- Java is an island in Indonesia where the first coffee was produced (called Java coffee). Java name was chosen by James Gosling while having a cup of coffee nearby his office
- The Java platform allows software to be developed and used across different architectures and operating systems



Overview

- The principles for creating Java programming language were "Simple, Robust, Portable, Platform-independent, Secured, High Performance, Multithreaded, Architecture Neutral, Object-Oriented, Interpreted, and Dynamic"
- Firstly, it was called "Greentalk" by James Gosling, and the file extension was .gt
- After that, it was called Oak and was developed as a part of the Green project that was started by small team of Sun Engineers (Called Green Team) in 1991.
- Oak is a symbol of strength and chosen as a national tree of many countries like the U.S.A., France, Germany, Romania, etc.

Java Versions

- Many java versions have been released till now. The current stable release of Java is Java SE 10

1. JDK Alpha and Beta (1995)	8. Java SE 6 (11th Dec 2006)	15. Java SE 13 (September 2019)
2. JDK 1.0 (23rd Jan 1996)	9. Java SE 7 (28th July 2011)	16. Java SE 14 (Mar 2020)
3. JDK 1.1 (19th Feb 1997)	10. Java SE 8 (18th Mar 2014)	17. Java SE 15 (September 2020)
4. J2SE 1.2 (8th Dec 1998)	11. Java SE 9 (21st Sep 2017)	18. Java SE 16 (Mar 2021)
5. J2SE 1.3 (8th May 2000)	12. Java SE 10 (20th Mar 2018)	19. Java SE 17 (September 2021)
6. J2SE 1.4 (6th Feb 2002)	13. Java SE 11 (September 2018)	20. Java SE 18 (March 2022)
7. J2SE 5.0 (30th Sep 2004)	14. Java SE 12 (March 2019)	

- Since Java SE 8 release, the Oracle corporation follows a pattern in which every even version is release in March month and an odd version released in September month



Java editions

- **Java Micro Edition (ME)** - designed for running Java applications on mobile devices with limited resources.
- **Java Standard Edition (SE)** - the general purpose version for desktop PCs and servers
- **Java Enterprise Edition (EE)** - SE plus some additional APIs for large enterprise server applications
- **JavaFX:** is an open-source, next-generation client application platform for desktop, mobile and embedded systems built on top of Java.



The platform

- **Java Runtime Environment (JRE)**

- This a virtual machine which runs programs which have been compiled
- Contains a large library of classes for lots of different purposes

- **Java Development Kit (JDK)**

- Contains tools such the compiler
- Has a copy of the JRE

A C program...

Source file(s)

```
int  
main()  
{  
}
```

Test.c



Compiler



gcc.exe



Machine code

```
0101110  
0010110  
1101010  
1010101
```

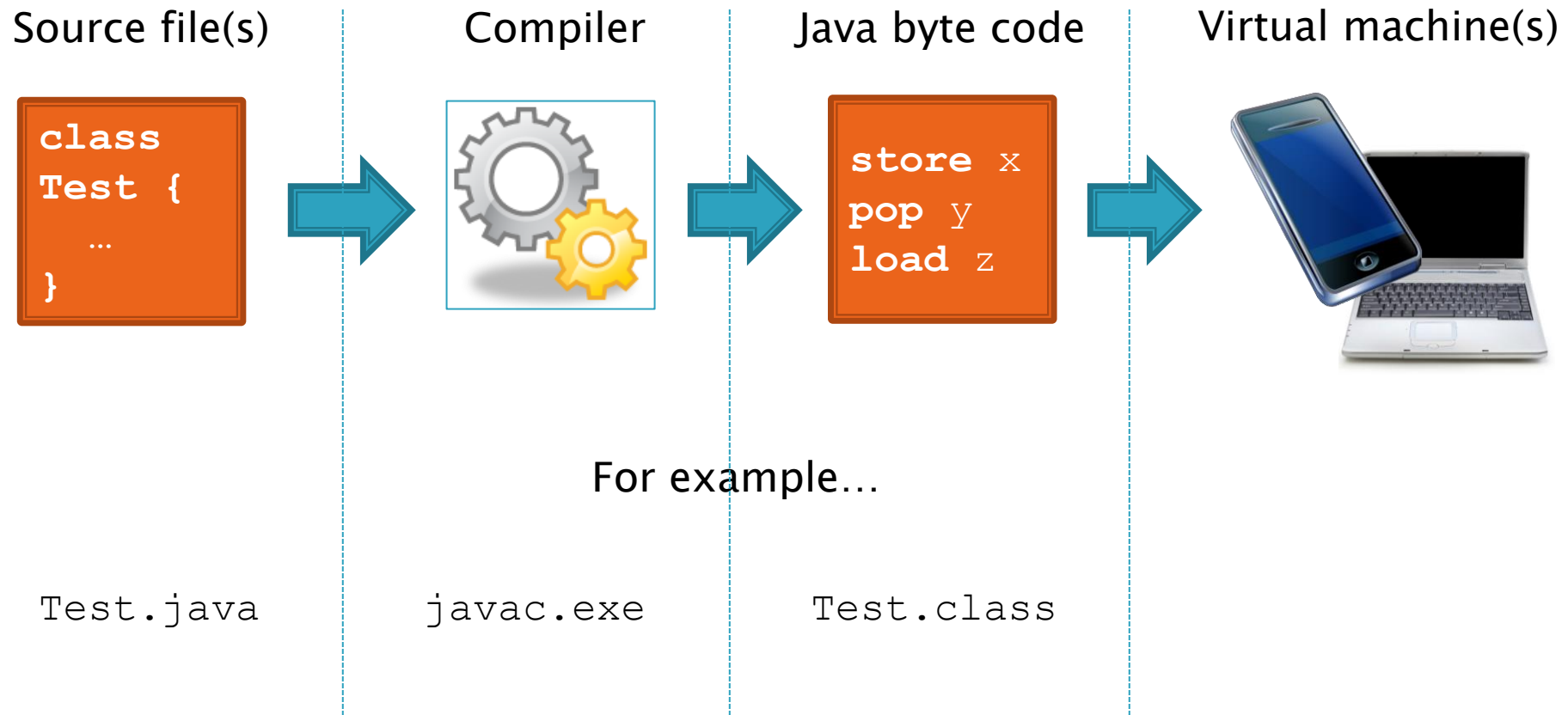
Text.exe
containing
x86 instructions



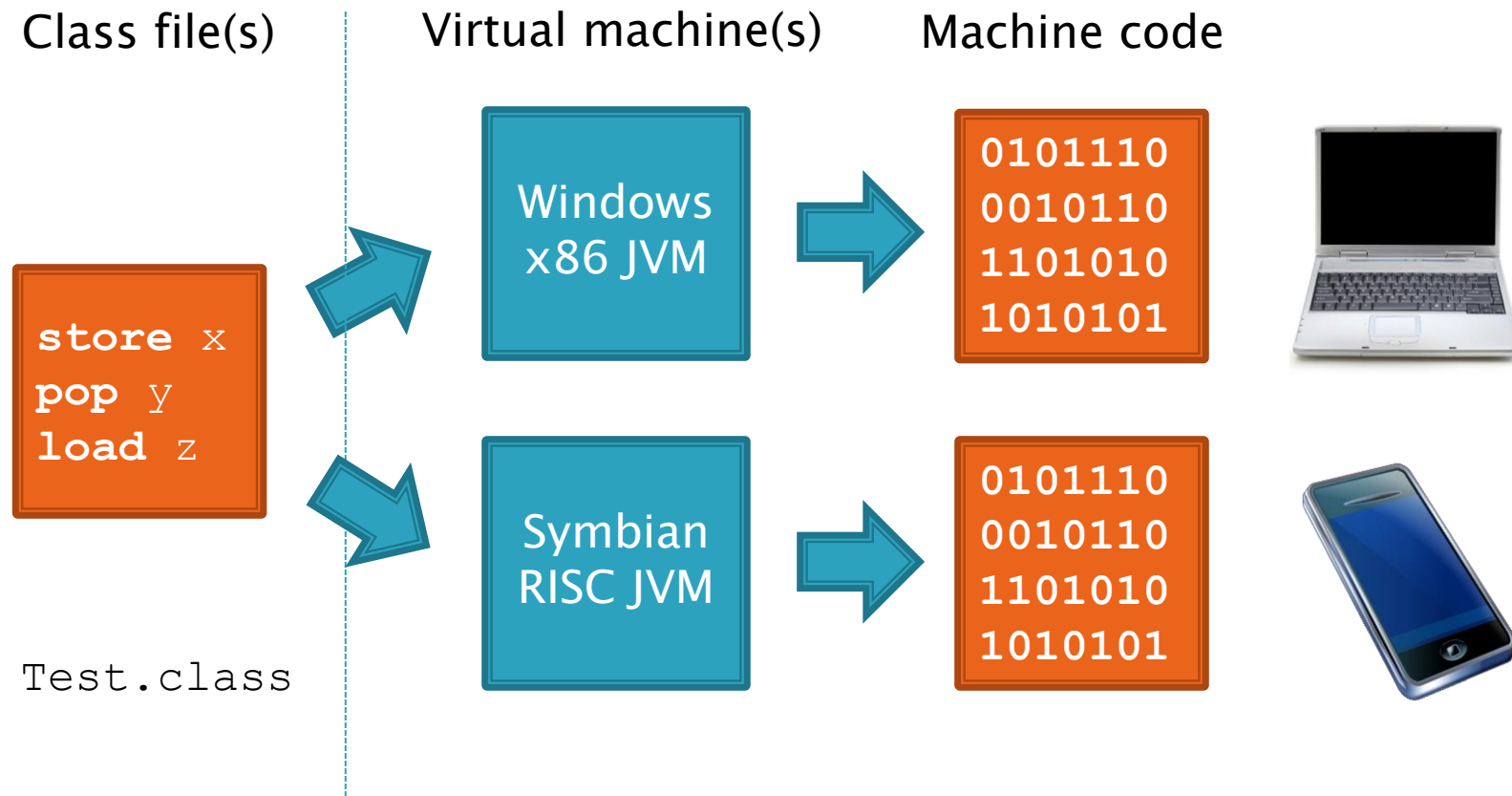
Runs on an x86
processor

For example...

A Java program...



Virtual machines



Example

```
class Greeting {  
    public static void main(String[] args){  
        System.out.println("Hello RCA Student!");  
    }  
}
```

```
$ javac Greeting.java
```



Greeting.class

```
$ java Greeting
```



```
$ Hello RCA Student!
```

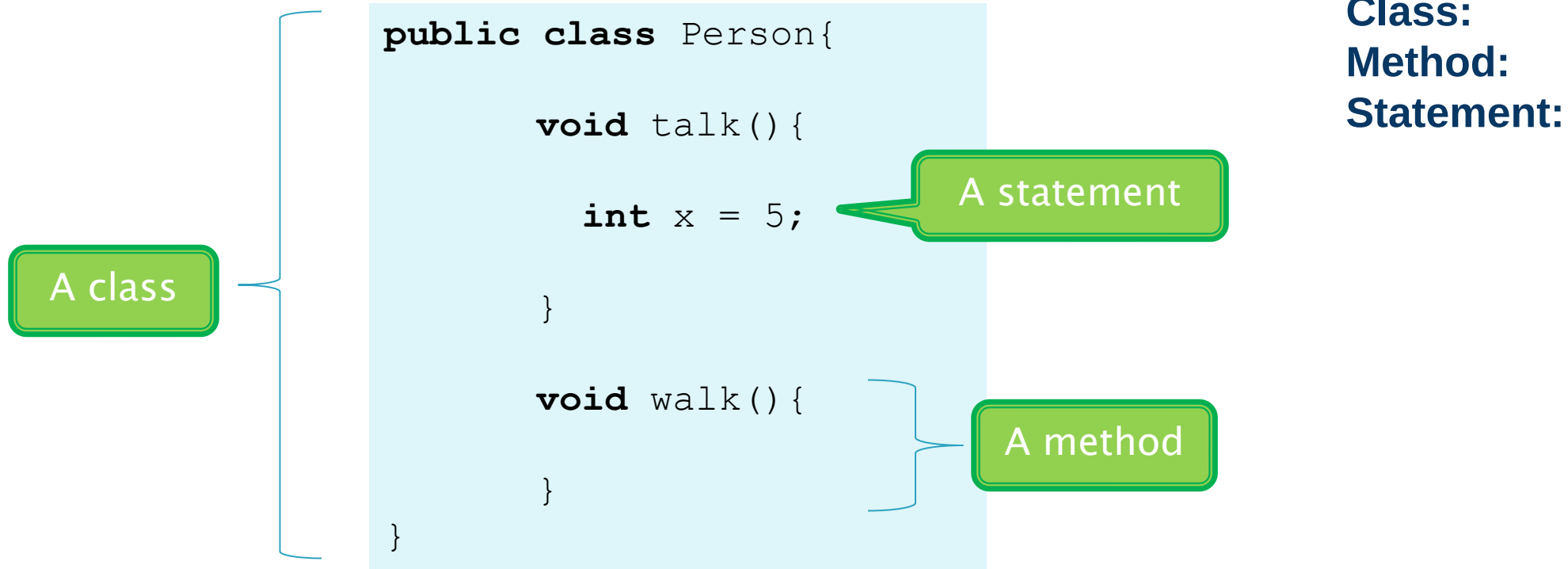
Write the
source file

Compile the
class file

Run the class
file in the
virtual
machine

View the
output

A Java source file



`public`, `class`, `void` and `int` are all Java keywords

More detail...

Access modifier

No return
value

Method
name

A method
argument

```
public class Greeting{  
    public static void main(String[] args){  
        System.out.println("Hello RCA Students!");  
    }  
}
```

Output text to
the console

Launch time

- Calling the `java` program launches the JVM
- You specify the name of the class and any arguments to send it, e.g.

```
$ java Greeting "Hello" 4
```

- The JVM searches the class for a method called `main`, and then calls that method
- It sends the arguments (e.g. `"Hello"` and `"4"`) as items in the array called `args`

Write your first class!

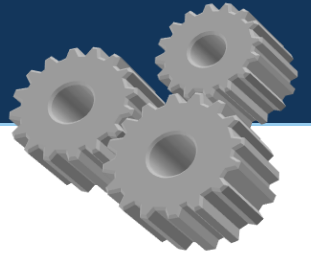
- Create a **.java** file
 - Define a class (should have the same name as the Java file)
 - Add a the main method

```
public static void main(String[] args) { ... }
```

- Call `System.out.println(...)` to print something
- Compile with **javac**
- Run the file with **java**



The compiler



- Converts the .java files to Java **bytecode**
- Reports any errors that prevented it from completing, or warnings that the developer should consider
 - Tells the developer what is wrong
 - Gives the source file and line number

Statements

- A statement does something, e.g.
 - Declaring a variable, `int x;`
 - Assigning a value to a variable, `x = 10;`
 - Incrementing a variable, `x++;`
 - Calling a method, `System.out.println("X");`
- Statements are separated by semi-colons, e.g.
 - `x = 10; y = x;`

Expressions

- An expression evaluates to a value, e.g.

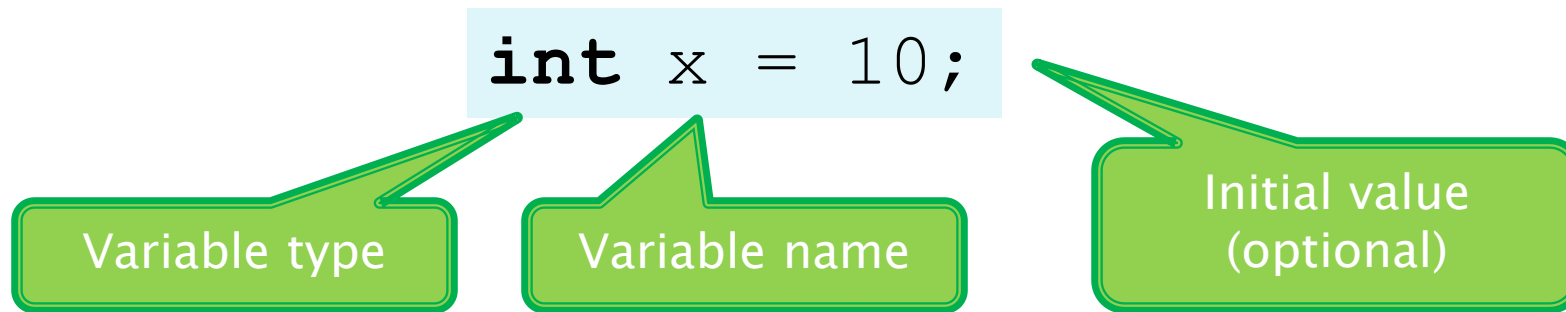
• <code>10 + 2 * 3 / 4</code>	→	11
• <code>"Hello" + " world"</code>	→	"Hello world"
• <code>3</code>	→	3
• <code>"kind of.."</code>	→	"kind of.."

- A statement can be an expression if it evaluates to a value, e.g.

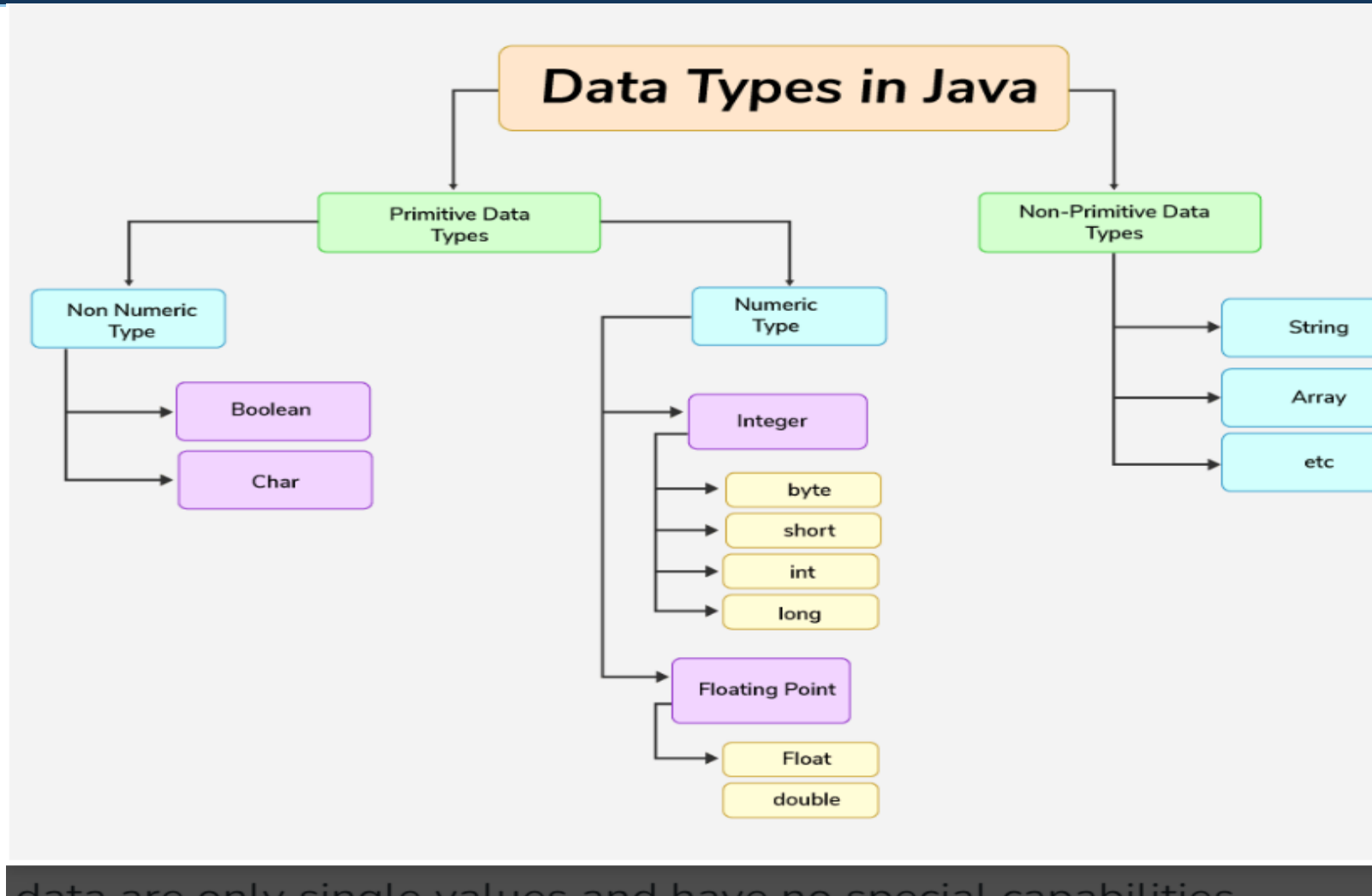
• <code>x = 10</code>	→	10	e.g. <code>y = (x = 10);</code>
• <code>Math.sin(5)</code>	→	-0.9589...	

Variables

- A value is stored in a variable so that it can be used elsewhere in a program, e.g.



- Variables can be **primitive types** or **object references**

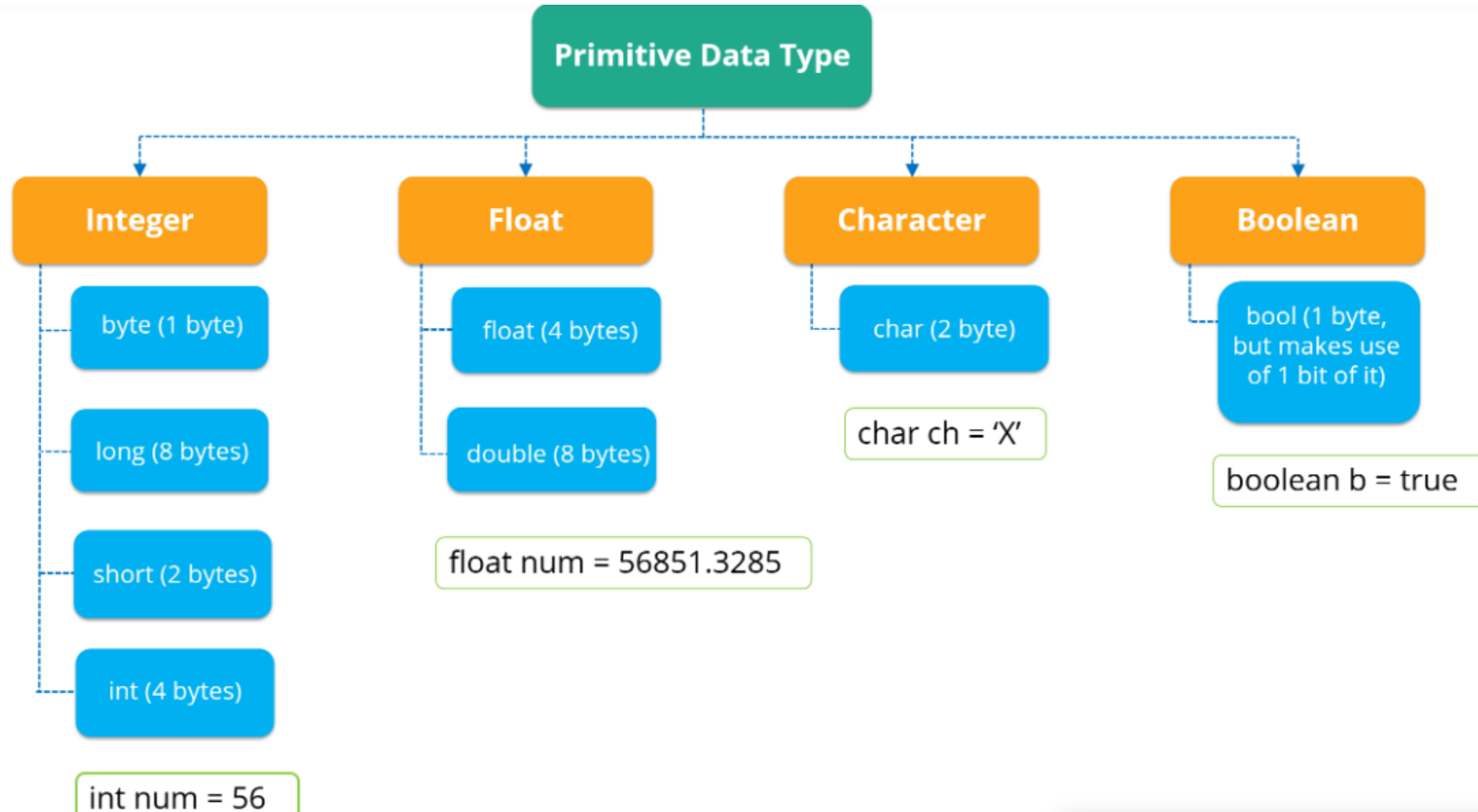


data are only single values and have no special capabilities

Primitive types

- A primitive data type is pre-defined by the programming language. The size and type of variable values are specified, and it has no additional methods

Primitive types



Primitive types

- You can generally assign the value of a smaller primitive type to a larger one, e.g.

```
short big = 5646;  
int bigger = big;  
long biggest = bigger;
```

- But not the other way around

```
long big = 3453434623426;  
int notSoBig = big;  
short evenSmaller = notSoBig;
```

Compiler error!

Casting primitives

- To assign a value to a smaller type, you have to use the cast operator, e.g.

```
long big = 3453;
```

```
int notSoBig1 = big;
```

Compiler error!

```
int notSoBig2 = (int)big;
```

```
short smaller = (short)big;
```

Works

Objects

- These are complex types which are defined in the JDK or in your code, e.g.

- `String` - a sequence of characters
- `Date` - a date and time value

- An object is created in memory using the **new** keyword, e.g.

```
String s1 = new String("Hello");
```

The object's
class

Reference
variable

The object
itself

Objects data types

Types that hold complex values (Objects)

String is a reference types

```
public static void main(String[] args) {  
    String s = "hello";  
}
```



The variable contains the address of the value.

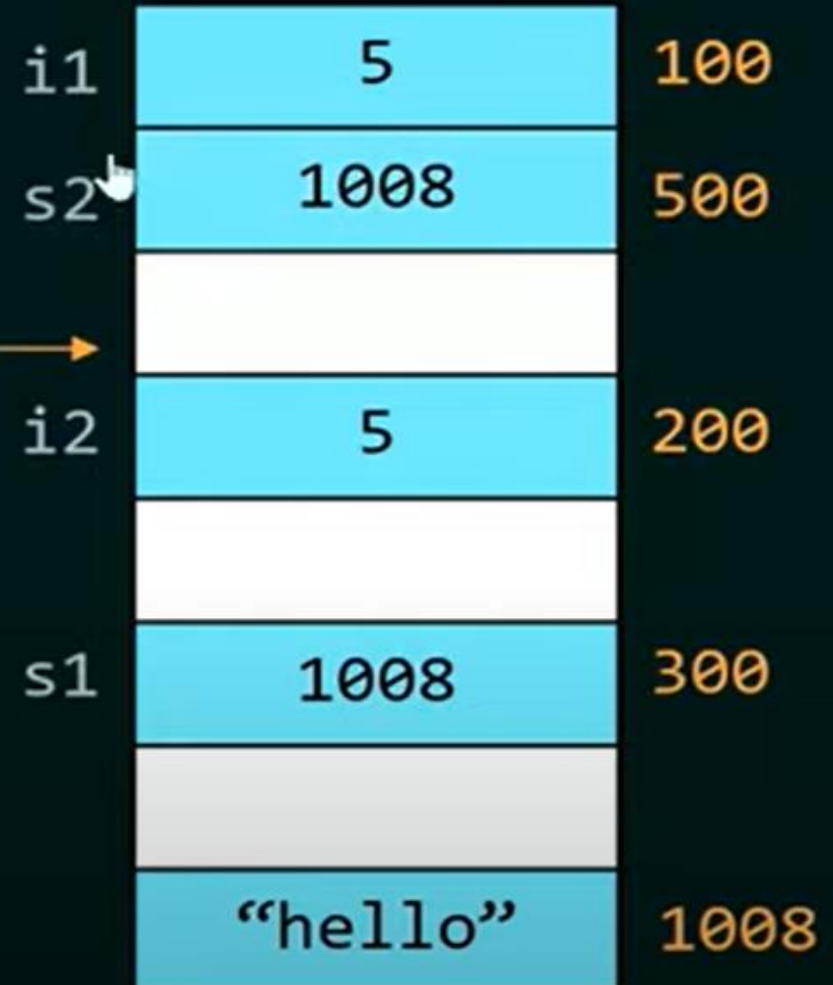
Primitive data types vs Reference data types

```
public static void main(String[] args) {  
    int i1 = 5;  
    int i2 = i1;  
  
    String s1 = "hello";  
    String s2 = s1;  
}
```

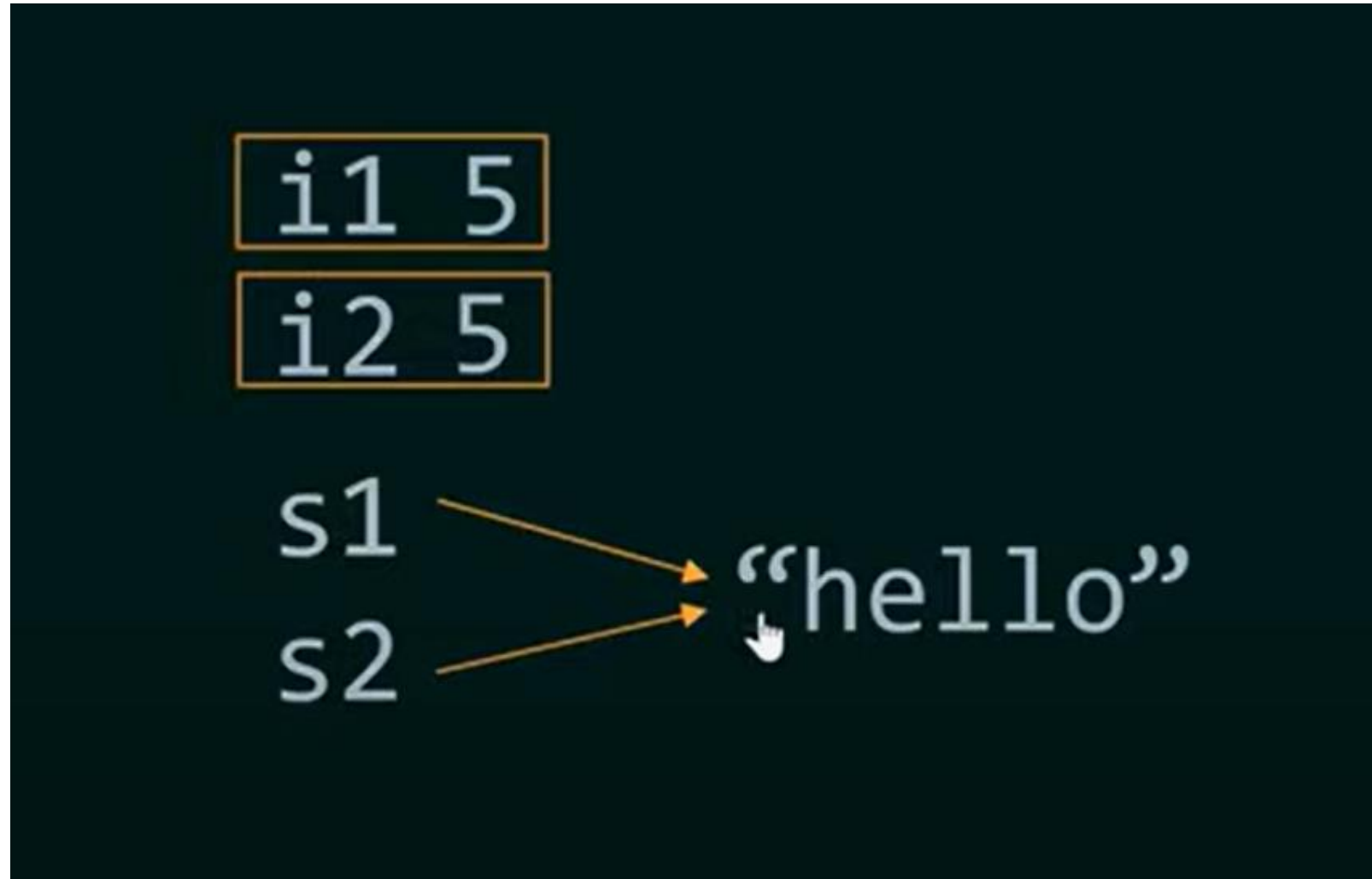
Primitive data types vs Reference data types

```
String s2 = s1;
```

Create a new variable s2
and assign the value of
s1 to it



Primitive data types vs Reference data types



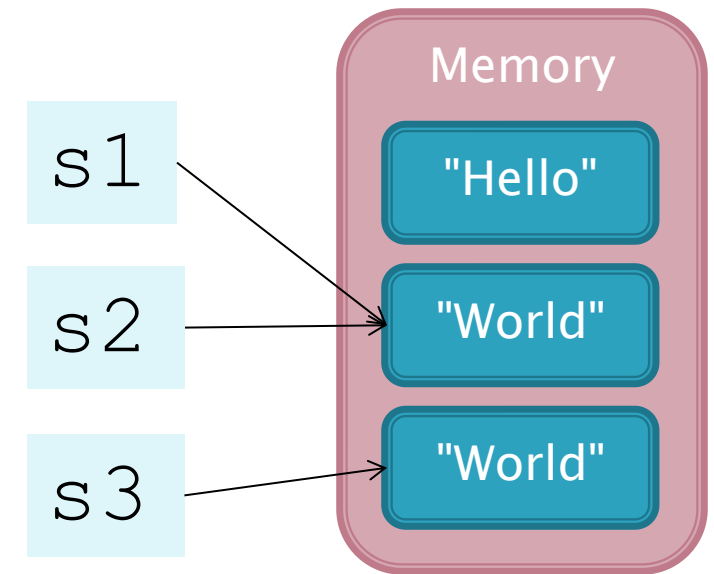
Equality

- Two types of equality...
- **Reference equality** `x == y`
 - Checks if the variables reference the same object in memory
- **Object equality** `x.equals(y)`
 - Checks if the objects which the variables reference are equal, i.e. have the same meaning or content
 - E.g. for Strings - do they have the same characters?

Equality example

```
String s3 = new String("World");
```

- `s1` and `s2` reference the same object so `s1 == s2` is **TRUE**
- `s1` and `s3` reference different objects so `s1 == s3` is **FALSE** even though the strings are the same

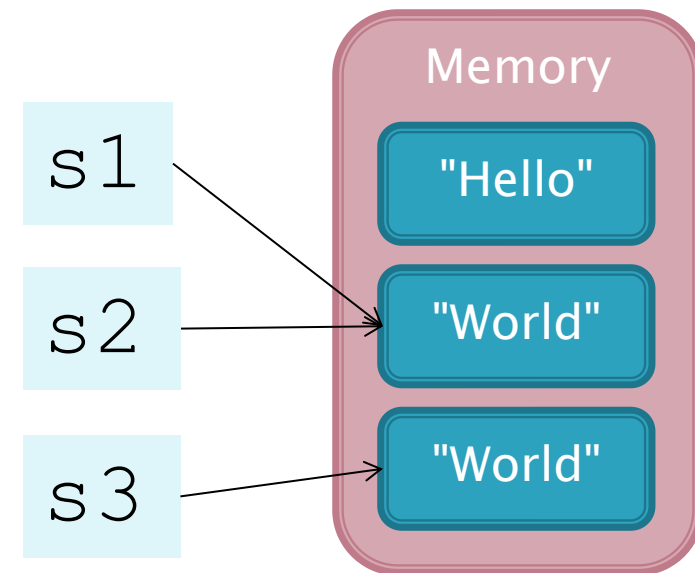


Equality example

- `s1` and `s3` reference `String` objects with the **same content** so `s1.equals(s3)` is **TRUE**

- Summary...

<code>s1 == s2</code>	TRUE
<code>s1 == s3</code>	FALSE
<code>s1.equals(s2)</code>	TRUE
<code>s1.equals(s3)</code>	TRUE



Naming conventions

- The compiler won't complain if you don't stick to these, but we will!

Entity	Convention	Examples
Class name	Camelcase, starts uppercase	HelloWorldProgram UserRole
Method name	Camelcase, starts lowercase	getAllStudents main
Variable	Camelcase, starts lowercase	numStudents listOfUsers
Constant	Uppercase with underscores	MAX_STUDENT_AGE DEFAULT_USER

1.2. Programming basics & Control Structures



Flow control: if-else

```
if (condition) {  
  
}  
else if (another condition) {  
  
}  
else {  
  
}
```

```
if (a == b) {  
    doMethod();  
}  
else if (isJava()) {  
    a = 10;  
}  
else {  
    out.println("X");  
}
```

```
if (a < x) {  
    a = 5;  
}
```



```
if (a < x)  
    a = 5;
```

If only one statement follows the if or else if, then the braces aren't needed

Flow control: while / do-while

```
while (condition) {  
    // loop these statements  
}
```

```
while (a < 10) {  
    out.println(a);  
    a++;  
}
```

```
do {  
    // loop these statements  
}  
while (condition);
```

do while means that the statements will always been executed at least once

Flow control: for

```
for (statement; condition; statement) {  
    // loop these statements  
}
```

```
for (int i = 0; i < 10; i++) {  
    System.out.println(i);  
}
```



```
int i = 0 ;  
while (i < 10) {  
    System.out.println(i);  
  
    i++;  
}
```

for is a alternative way
to write a while loop

Flow control: continue/break

```
for (int i = 0; i < 10; i++) {
```

```
    if (i == x)
```

```
        break;
```

```
    else if (i == z)
```

```
        continue;
```

```
    System.out.println(i);
```

```
}
```

break causes the for loop to finish immediately

continue goes to the next iteration if there will be one

Flow control: switch

```
switch (variable) {  
  case <value1>:  
    statements  
    break;  
  case <value2>:  
    statements  
    break;  
  default:  
    statements  
}
```



```
switch (choice) {  
  case 'Y':  
    doThing();  
    break;  
  case 'N':  
    exitProgram();  
    break;  
  default:  
    showHelp();  
}
```

switch can work with byte,
char, short and int values

Flow control: switch

```
switch (choice) {  
  case 'Y':  
    doThing() ;  
    break ;  
  case 'N':  
    exitProgram() ;  
    break ;  
  default:  
    showHelp() ;  
}
```



```
if (choice == 'Y')  
    doThing() ;  
else if (choice == 'N')  
    exitProgram() ;  
else  
    showHelp() ;
```

switch is often a better
way of writing an if-else
statement

Flow control: switch

```
switch (choice)
{
  case 'Y':
    doThing() ;
  case 'N';
  case 'X';

  exitProgram() ;
  break;
default:
  showHelp() ;
}
```

What happens now
when choice equals 'Y'?

What about 'X'?

Further reading

- <http://java.sun.com/docs/books/tutorial/java/nutsandbolts/index.html>
- <https://docs.oracle.com/en/java/javase/19/language/java-language-changes.html>
- [History of Java – Javatpoint](#)

EoF

